

Task 1 : Stream and Pipeline Processing – Group Anagrams

Aim

To develop a Java program that groups words that are anagrams of each other using the Java Stream API and pipeline processing.

Algorithm

1. Start the program.
2. Create an array or list of strings.
3. Convert the array into a stream using `Arrays.stream()`.
4. For each word:
 - o Convert it into a character array.
 - o Sort the characters.
 - o Convert the sorted characters back into a string.
5. Use the sorted string as the key to group the original words using `Collectors.groupingBy()`.
6. Collect the grouped words into a map.
7. Display each group of anagrams.
8. Stop the program.

Program (Java 21)

```
import java.util.*;
```

```
import java.util.stream.*;
```

```
public class GroupAnagrams {
```

```
    public static void main(String[] args) {
```

```
        String[] words = {"eat", "tea", "tan", "ate", "nat", "bat"};
```

```
        Map<String, List<String>> grouped = Arrays.stream(words)
```

```
        .collect(Collectors.groupingBy(word -> {
            char[] ch = word.toCharArray();
            Arrays.sort(ch);
            return new String(ch);
        }));

        System.out.println("Grouped Anagrams:");
        grouped.values().forEach(System.out::println);
    }
}
```

Output

Grouped Anagrams:

[eat, tea, ate]

[tan, nat]

[bat]

Note: The order of the groups may vary because HashMap does not guarantee insertion order.

Result

The Java program successfully grouped the given words into sets of anagrams using Stream API and pipeline processing. It demonstrates efficient data processing through stream operations and the `Collectors.groupingBy()` method.

Task 2 : Stream and Pipeline Processing – Top K Frequent Elements

Aim

To develop a Java program that finds the Top K Frequent Elements from an array using the Java Stream API and pipeline processing.

Algorithm

1. Start the program.
2. Create an array of integers.
3. Set the value of K (number of most frequent elements).
4. Convert the array into a stream using `Arrays.stream()`.
5. Group the elements and count their frequency using `Collectors.groupingBy()`.
6. Convert the frequency map into a stream.
7. Sort the entries in descending order based on frequency.
8. Select the first K elements using `limit(K)`.
9. Collect the results into a list.
10. Display the Top K frequent elements.
11. Stop the program.

Program (Java 21)

```
import java.util.*;  
  
import java.util.function.Function;  
  
import java.util.stream.Collectors;  
  
public class TopKFrequentElements {  
  
    public static void main(String[] args) {  
  
        int[] nums = {1, 1, 1, 2, 2, 3};  
        int k = 2;
```

```
List<Integer> result = Arrays.stream(nums)
    .boxed()
    .collect(Collectors.groupingBy(
        Function.identity(),
        Collectors.counting()))
    .entrySet()
    .stream()
    .sorted((a, b) -> Long.compare(b.getValue(), a.getValue()))
    .limit(k)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

System.out.println("Top " + k + " Frequent Elements: " + result);
}
}
```

Output

Top 2 Frequent Elements: [1, 2]

Result

The Java program successfully identified the Top K Frequent Elements using the Java Stream API and pipeline processing. The program grouped elements, counted their frequencies, sorted them in descending order, and displayed the top K most frequent elements efficiently.

Task 3 : Stream API & Collection Processing – Java Deque

Aim

To develop a Java program that demonstrates the use of the Deque collection by performing insertion, deletion, and traversal operations.

Algorithm

1. Start the program.
2. Create a Deque using ArrayDeque.
3. Insert elements at the front and rear using addFirst() and addLast().
4. Display the deque.
5. Remove an element from the front using removeFirst().
6. Remove an element from the rear using removeLast().
7. Display the updated deque.
8. Traverse and print all elements.
9. Stop the program.

Program (Java 21)

```
import java.util.ArrayDeque;
```

```
import java.util.Deque;
```

```
public class JavaDequeDemo {
```

```
    public static void main(String[] args) {
```

```
        Deque<String> deque = new ArrayDeque<>();
```

```
        // Insert elements
```

```
        deque.addFirst("Apple");
```

```
        deque.addLast("Banana");
```

```
deque.addFirst("Mango");
deque.addLast("Orange");

System.out.println("Deque after insertion:");
System.out.println(deque);

// Remove elements
deque.removeFirst();
deque.removeLast();

System.out.println("Deque after deletion:");
System.out.println(deque);

// Traverse the deque
System.out.println("Elements in Deque:");
for (String fruit : deque) {
    System.out.println(fruit);
}
}
```

Output

Deque after insertion:

[Mango, Apple, Banana, Orange]

Deque after deletion:

[Apple, Banana]

Elements in Deque:

Apple

Banana

Result

The Java program successfully demonstrated the use of the Deque collection. It performed insertion and deletion operations at both ends of the deque and displayed the remaining elements, illustrating efficient collection processing using the Java Collections Framework.

Task 4 : Stream API & Collection Processing – Java HashSet

Aim

To develop a Java program that demonstrates the use of the HashSet collection to store unique elements and perform basic operations.

Algorithm

1. Start the program.
2. Create a HashSet of strings.
3. Add elements to the HashSet.
4. Attempt to add duplicate elements.
5. Display the contents of the HashSet.
6. Check whether a specific element exists using contains().
7. Remove an element using remove().
8. Display the updated HashSet.
9. Stop the program.

Program (Java 21)

```
import java.util.HashSet;

public class JavaHashSetDemo {

    public static void main(String[] args) {

        HashSet<String> set = new HashSet<>();

        // Add elements
        set.add("Apple");
        set.add("Banana");
        set.add("Mango");
```



```
set.add("Orange");
set.add("Apple"); // Duplicate element

System.out.println("HashSet Elements:");
System.out.println(set);

// Check element
System.out.println("Contains Mango: " + set.contains("Mango"));

// Remove an element
set.remove("Banana");

System.out.println("HashSet after removing Banana:");
System.out.println(set);
}
}
```

Output

HashSet Elements:

[Apple, Banana, Mango, Orange]

Contains Mango: true

HashSet after removing Banana:

[Apple, Mango, Orange]

Note: The order of elements in a HashSet is not guaranteed and may vary each time the program is executed.

Result

The Java program successfully demonstrated the use of the HashSet collection. It stored unique elements, ignored duplicate values automatically, checked for the presence of an element, removed an element, and displayed the updated set.